VOL. 21

Java

The language, the core APIs and the build tools behind the JVM ecosystem

Source	docs.oracle.com/javase/tutorial
Generated	2026-10-10
Level	Intermediate
Estimated study time	35 hours
Language	English
Volume	Vol. 21 - Java

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 3

Chapter map . . . 3

Chapter 1 - The JDK, the JVM and your first program . . . 4

1.1 Summary . . . 4

1.2 Concepts . . . 5

JDK, JRE, JVM . . . 5

From source to running program . . . 5

Two ways to run a single file . . . 5

JShell . . . 6

1.3 Examples . . . 6

1.4 Common mistakes and best practices . . . 8

1.5 Practice exercises . . . 8

Solutions . . . 9

1.6 Mini test . . . 9

Answer key and explanations . . . 9

Chapter 2 - Types, variables and operators . . . 10

2.1 Summary . . . 10

2.2 Concepts . . . 10

The eight primitives . . . 10

Stack and heap . . . 11

Promotion and casting . . . 11

Strings . . . 11

Text blocks . . . 11

2.3 Examples . . . 11

2.4 Common mistakes and best practices . . . 14

2.5 Practice exercises . . . 15

Solutions . . . 15

2.6 Mini test . . . 15

Answer key and explanations . . . 15

How to use this volume

Prerequisites. You can write a few lines of code in any language, you know what a variable, a function and a loop are, and you are comfortable using a terminal. No previous Java experience is assumed: chapter 1 installs everything you need on a Windows, macOS or Linux machine.

Estimated study time. About 35 hours: two to three hours per chapter, plus the time you spend breaking the examples on purpose. Chapters 1 to 8 are the language core and must be read in order. Chapters 9 to 12 are the API core and can be read in any order. Chapters 13 and 14 are about packaging and tooling and pay off as soon as you start a real project.

How the chapters work. Every chapter opens with a summary of the ideas, explains the concepts with short tables and ASCII diagrams, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and closes with practice exercises, their solutions, and a mini test with an answer key. Read a chapter, then retype the examples by hand: typing beats reading.

How to run the code. Every program is written for Java 17 LTS (the syntax also runs unchanged on Java 21 and later). Since JDK 11 the `java` launcher can compile and run a single source file in memory, so most examples need no build tool at all:

shell

```
java Hello.java                 # compile in memory and run
javac -d out Hello.java         # compile to out/Hello.class
java -cp out Hello              # run the class file
jshell                          # interactive scratchpad
```

output

```
Hello, Java!
```

A note about the output blocks. The machine this volume was written on had no JDK installed (`java -version` and `javac -version` both report that the command is unknown), so no example in this book was executed. Every `text` block is the expected output, derived by hand from the Java Language Specification and the API documentation; where a number or an ordering depends on the machine, the example is written so that it cannot. Treat the outputs as a prediction to check against your own runs, and expect your build numbers and temp file names to differ.

About the source. The syllabus follows the official Java tutorial published by Oracle at docs.oracle.com/javase/tutorial, with modern material cross-checked against dev.java and the Java 17 API documentation. The Oracle tutorial is written for JDK 8 and is openly licensed for reuse in places; this volume is original writing that follows its topic order and cites its pages as sources.

Chapter map

#	Chapter	You will learn
1	The JDK, the JVM and your first program	what the JDK contains, bytecode, `java Hello.java`, JShell
2	Types, variables and operators	primitive types, `var`, `final`, promotion, casting, text blocks
3	Control flow	`if`, `switch` expressions, loops, labels, `break`/`continue`
4	Classes and objects	fields, constructors, encapsulation, records, enums, statics
5	Inheritance, interfaces and polymorphism	`extends`, `@Override`, interfaces with default methods, sealed types
6	Generics	type parameters, bounded types, wildcards, erasure, PECS
7	The Collections Framework	`List`, `Set`, `Map`, `Deque`, sorting, immutable factories
8	Exceptions	checked versus unchecked, try-with-resources, custom exceptions
9	Functional interfaces and lambdas	`java.util.function`, method references, closures
10	Streams	pipelines, lazy evaluation, collectors, `Optional`
11	File I/O and NIO.2	`Path`, `Files`, readers and writers, walking a tree
12	Concurrency basics	threads, executors, `Future`, visibility, `volatile`, atomics
13	Modules (JPMS) and packaging	`module-info.java`, `jar`, `jdeps`, `jlink`
14	Maven, Gradle and tooling	`pom.xml`, Gradle Kotlin DSL, JUnit 5, everyday tools
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - The JDK, the JVM and your first program

1.1 Summary

- Java source code is compiled into bytecode, an instruction set the Java Virtual Machine executes, which is why one class file runs on every platform that has a JVM.
- The JDK is the developer kit: it ships the compiler `javac`, the launcher `java`, the archiver `jar`, the shell `jshell` and a dozen diagnostic tools.
- Since JDK 11 the `java` launcher can compile and run a single `.java` file in memory, which removes all build configuration from the first hour of learning.

- A source file may contain several top-level classes, but only one can be `public`, and the file must be named after it.
- `javac -d <dir>` decides where class files land, and `--class-path` tells the runtime where to find them.
- Bytecode is not machine code: the JVM interprets and JIT-compiles it, so a program keeps getting faster while it runs.
- JShell, added in JDK 9, gives you a read-eval-print loop where each statement answers immediately.
- The JRE (runtime only) no longer ships as a separate distribution since JDK 9; the JDK is what you install.
- `java -version` prints to standard error, while `java --version` prints to standard output, which matters in scripts.
- A Java program is a class with a `public static void main(String[] args)` method; the array carries the command line arguments.

1.2 Concepts

JDK, JRE, JVM

Three names get mixed up constantly. Keep them apart:

Term	What it is	Who needs it
JVM	the virtual machine that executes bytecode	runs the program
JRE	a JVM plus the core class libraries (historical)	was enough to run, not to compile
JDK	the JRE plus `javac`, `jar`, `jshell`, debuggers	what you install

From source to running program

output

```
 Hello.java            javac              Hello.class          java             process
+-------------+   ---------------->   +--------------+   ------------->   +-----------+
| public class|   compile to bytecode | magic CAFEBABE|   JVM loads it   | Hello,    |
| Hello {...} |                       | constant pool |   JIT compiles   | Java!     |
+-------------+                       +--------------+                  +-----------+
```

Compilation is a separate, explicit step. Errors found by `javac` are compile time errors; everything that happens after is a runtime matter.

Two ways to run a single file

The traditional way is two commands: `javac Hello.java` then `java Hello`. Since JDK 11 there is a shortcut, the source-file launcher: `java Hello.java` compiles the class in memory, runs the first `main` it finds, and never writes a class file to disk. It is ideal for learning and for small scripts.

Rules that bite in the first week:

Rule	Detail
File name	`public class Hello` must live in `Hello.java`
Case	class names are case sensitive; `hello` is not `Hello`
Entry point	`public static void main(String[] args)`
Package	no `package` line means the default (unnamed) package
Arguments	`args[0]` is the first word after the class name

JShell

`jshell` starts a shell inside the JVM. Declarations stay alive between lines, so you can build a class incrementally:

shell

```
$ jshell
|  Welcome to JShell -- Version 17.0.10
|  For an introduction type: /help intro

jshell> int total = 0;
total ==> 0

jshell> for (int i = 1; i <= 5; i++) total += i;
total ==> 10

jshell> total * 2
$3 ==> 20
```

`/list`, `/vars`, `/methods`, `/imports`, `/open file.java` and `/exit` are the commands you will actually use.

1.3 Examples

Outputs in this chapter were not executed on the authoring machine; they were derived by hand from the JDK 17 behaviour. Version numbers will differ on your machine.

Example 1: check which runtime you are talking to.

shell

```
java --version
javac -version
```

output

```
java 17.0.10 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 17.0.10+7)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.10+7)
javac 17.0.10
```

Example 2: the classic first program, run straight from source.

Java

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

output

```
Hello, Java!
```

Example 3: the same program, compiled first.

shell

```
javac -d out Hello.java
java -cp out Hello
ls out
```

output

```
Hello, Java!
Hello.class
```

Example 4: read the command line.

Java

```java
public class Greet {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("nobody to greet");
            return;
        }
        System.out.println("Hello, " + args[0]);
    }
}
```

shell

```
java Greet.java Ada
java Greet.java
```

output

```
Hello, Ada
nobody to greet
```

Example 5: a class with a helper method and a package-less layout.

Java

```java
public class Circle {
    static double area(double radius) {
        return Math.PI * radius * radius;
    }

    public static void main(String[] args) {
        for (double r : new double[] {1.0, 2.5}) {
            System.out.println("r=" + r + " area=" + area(r));
        }
    }
}
```

output

```
r=1.0 area=3.141592653589793
r=2.5 area=19.634954084936208
```

`Math.PI` is the closest `double` to pi, so the digits stop after sixteen significant figures, not where you might expect.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Saving as `hello.java` for `public class Hello`	`javac` refuses the file	match the public class name exactly
Calling `java Hello.class`	the launcher treats the argument as a class name	`java Hello`, or `java Hello.java`
Mixing `java` and `javac` versions	class file version errors at runtime	keep one JDK 17 on `PATH`
Comparing strings with `==`	compares references, not characters	use `equals` (chapter 2)
Ignoring the compile error line number	the message names file and line	read it top to bottom; the first error is the real one
Writing everything in `main`	nothing is testable or reusable	extract methods and classes early

- When a command is not found, check `PATH` before reinstalling anything.
- Keep one directory per project, even for a single file.

1.5 Practice exercises

1. Write a program that prints the number of arguments it received followed by each argument on its own line, and run it twice with different arguments.
2. Explain the difference between `java Hello` and `java Hello.java`.
3. Which file name must contain `public class Toolbox`?
4. Rewrite example 5 so that `main` reads the radius from `args[0]` and prints one line.
5. What does `javac -d build X.java` do, and what command follows it?

Solutions

1. `java public class Args { public static void main(String[] args) { System.out.println(args.length); for (String arg : args) { System.out.println(arg); } } }` `java Args.java a b` prints `2`, then `a`, then `b`.
2. `java Hello` loads the already compiled `Hello.class` from the class path; `java Hello.java` compiles the source in memory first and runs it directly.
3. `Toolbox.java` - the file name must match the public class name.
4. `double r = Double.parseDouble(args[0]);` then `System.out.println(area(r));`, with a guard for a missing argument.
5. It compiles `X.java` and writes the class files under `build/`, keeping the package directories; afterwards `java -cp build X` runs it.

1.6 Mini test

1. What does the Java compiler produce? A) Machine code for the current CPU B) Bytecode for the JVM C) An executable with a `.exe` extension D) JavaScript
2. Which command compiles `Main.java` into a `build` directory? A) `java Main.java -d build` B) `javac -d build Main.java` C) `javac Main.java build` D) `jar -d build Main.java`
3. True or false: since JDK 11 you can run a single source file with `java File.java` without producing a class file on disk.
4. Which signature does the JVM look for as the entry point? A) `static void main()` B) `public static void main(String[] args)` C) `public void run(String[] args)` D) `static int main(String args)`
5. Where do the command line arguments arrive? A) In `System.getProperties()` B) In the `args` array of `main` C) On standard input D) In environment variables only
6. What is JShell? A) A build tool B) A JVM profiler C) An interactive read-eval-print loop D) A package manager
7. Why does the same class file run on Windows, macOS and Linux? A) The compiler produces three versions B) Each operating system has a JVM that interprets and compiles the bytecode C) Java uses the operating system's bytecode D) It does not, Java is not portable

Answer key and explanations

1. B - `javac` emits platform independent bytecode stored in a `.class` file that starts with the magic number `CAFEBABE`.
2. B - `-d` sets the destination directory for the generated class files.
3. True - the source-file launcher compiles in memory and runs the class.
4. B - the JVM invokes that exact signature; `String... args` also works because varargs compile to the same array.
5. B - `args` holds the words written after the class name.
6. C - JShell keeps declarations between lines so you can experiment.
7. B - the JVM is the platform-specific part; bytecode is the portable one.

Chapter 2 - Types, variables and operators

2.1 Summary

- Java has eight primitive types: four integer types, two floating point types, `char` and `boolean`.
- Everything that is not a primitive is an object living on the heap, reached through a reference.
- `int` is the default choice for whole numbers and `double` for decimals; `long` and `float` are for size or memory pressure.
- Literals support underscores as digit separators and suffixes `L`, `f`, `d` to force a type.
- `var` asks the compiler to infer the type of a local variable from the initialiser, but the variable is still statically typed.
- `final` makes a variable single assignment, which is how you get constants that the compiler can reason about.
- Primitive arithmetic promotes operands to `int` or wider, so `byte + byte` is an `int`.
- Narrowing conversions need an explicit cast and can lose information silently.
- `String` is immutable, methods return new strings, and references are compared with `==` only for identity.
- Text blocks, final in Java 15, give multi-line strings without escape noise.

2.2 Concepts

The eight primitives

Type	Size	Range or precision	Example
`byte`	8 bit	-128 to 127	`byte b = 120;`
`short`	16 bit	-32768 to 32767	`short s = 30_000;`
`int`	32 bit	about -2.1e9 to 2.1e9	`int n = 8_100_000_000;`
`long`	64 bit	about -9.2e18 to 9.2e18	`long l = 9_000_000_000_000L;`
`float`	32 bit IEEE 754	about 7 decimal digits	`float f = 3.14f;`
`double`	64 bit IEEE 754	about 15 decimal digits	`double d = 3.14;`
`char`	16 bit UTF-16 code unit	`'A'` to `'□'`	`char c = 'A';`
`boolean`	not specified	`true` or `false`	`boolean ok = true;`

Every primitive has a wrapper class in `java.lang` (`Integer`, `Double`, `Character`, `Boolean`), and the compiler converts between them automatically. That convenience is called autoboxing; it costs an object allocation and a possible `NullPointerException`.

Stack and heap

output

```
  main's frame                heap
+----------------+       +------------------------+
| int count = 3  |       | String "Ada" (immutable)|
| String name ---+------>| char[] value            |
+----------------+       +------------------------+
```

A local primitive lives in the frame and dies with it. A local reference lives in the frame but points to an object that the garbage collector removes once nothing reachable refers to it.

Promotion and casting

Expression	Result type	Why
`byte + byte`	`int`	integer operands below `int` are promoted
`int + long`	`long`	the wider type wins
`int / int`	`int`	division truncates
`int / double`	`double`	one operand is `double`
`char + int`	`int`	`char` is an integer type
`(int) 3.99`	`int`	explicit narrowing truncates

Comparing a `double` with `==` is almost always a mistake: 0.1 + 0.2 is not 0.3 in binary floating point. Use a tolerance for comparisons.

Strings

`String` is a class, not a primitive, and it is immutable.

output

```
"Ada" + " " + "Lovelace"   ->  a new String object, nothing is modified
"ada".equals("Ada")        ->  false, content comparison
"ada".equalsIgnoreCase(..) ->  true
"Ada" == "Ada"             ->  true only if the compiler pooled both literals
new String("Ada") == "Ada" ->  false, different objects
```

Use `equals` for content, `==` for identity. Overriding `equals` obliges you to override `hashCode` (chapter 5).

Text blocks

A text block starts with `"""` and ends with a `"""`. The closing delimiter's column decides how much common indentation is removed:

Java

```java
String card = """
        {"name": "Ada", "year": 1843}""";
```

2.3 Examples

Outputs in this chapter were not executed on the authoring machine; they were derived by hand from Java 17 semantics.

Example 1: the primitives and their printed forms.

Java

```java
public class Primitives {
    public static void main(String[] args) {
        byte small = 120;
        short year = 2026;
        int population = 8_100_000_000;
        long stars = 9_000_000_000_000L;
        float pi = 3.14159f;
        double root = Math.sqrt(2);
        char grade = 'A';
        boolean ready = true;

        System.out.println(small + " " + year + " " + population);
        System.out.println(stars + " " + pi + " " + root);
        System.out.println(grade + " " + (int) grade + " " + ready);
        System.out.println(Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
    }
}
```

output

```
120 2026 8100000000
9000000000000 3.14159 1.4142135623730951
A 65 true
-2147483648 .. 2147483647
```

Example 2: `var` and `final`.

Java

```java
import java.util.List;

public class Inference {
    public static void main(String[] args) {
        final int limit = 3;
        var names = List.of("Ada", "Linus", "Grace");
        var total = 0;
        for (var name : names) {
            if (name.length() > limit) {
                total += name.length();
            }
        }
        System.out.println(names + " -> " + total);
    }
}
```

output

```
[Ada, Linus, Grace] -> 10
```

`var` cannot be used without an initialiser, for fields, or for method parameters; it only shortens the declaration.

Example 3: operators, precedence and the two divisions.

Java

```java
public class Operators {
    public static void main(String[] args) {
        int a = 7, b = 3;
        System.out.println(a + b + " " + (a - b) + " " + a * b + " " + a / b + " " + a % b);
        System.out.println(7 / 3.0);
        System.out.println((double) (a / b));

        int c = 5;
        c += 2;
        c *= 3;
        c++;
        System.out.println(c);

        System.out.println(a > b ? "bigger" : "smaller");
        System.out.println(1 + 2 + "x" + 1 + 2);
    }
}
```

output

```
10 4 21 2 1
2.3333333333333335
2.0
22
bigger
3x12
```

`+` with a `String` on either side concatenates, so `1 + 2 + "x"` evaluates `1 + 2` first; after the first string appears, every other `+` is concatenation.

Example 4: strings and a text block.

Java

```java
public class Text {
    public static void main(String[] args) {
        String name = "Ada";
        System.out.println(name + " has " + name.length() + " letters");
        System.out.println(name.indexOf("d") + " " + name.substring(1));
        System.out.println(name.equals("ada") + " " + name.equalsIgnoreCase("ada"));

        String card = """
                {"name": "Ada", "year": 1843}""";
        System.out.println(card.length() + " " + card.indexOf("\"year\""));
    }
}
```

output

```
Ada has 3 letters
2 da
false true
31 10
```

`card` contains no line break: the single line of content is `{"name": "Ada", "year": 1843}`, which is 31 characters, and `"year"` starts at index 10.

Example 5: promotion, narrowing and the classic integer division trap.

Java

```java
public class Casting {
    public static void main(String[] args) {
        byte a = 10;
        byte b = 20;
        int sum = a + b;
        System.out.println("sum=" + sum);

        long budget = sum;
        int back = (int) budget;
        System.out.println(back);

        double big = 3.99;
        System.out.println("cut=" + (int) big);

        char letter = 'B';
        int position = letter - 'A';
        System.out.println((char) ('A' + position) + " at " + position);

        System.out.println(9 / 2 + " vs " + 9 / 2.0 + " vs " + 9 % 2);
    }
}
```

output

```
sum=30
30
cut=3
B at 1
4 vs 4.5 vs 1
```

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`if (text == "quit")`	compares references, the branch never runs	`text.equals("quit")`, or `"quit".equals(text)` to be null safe
`1 / 2` expecting 0.5	integer division truncates first	make one operand `double`
`byte + byte` into a `byte`	the result is `int` and will not compile	assign to `int` or cast explicitly
Comparing doubles with `==`	rounding makes equal values unequal	compare with a tolerance such as `Math.abs(a - b) < 1e-9`
Money in `double`	0.1 cannot be represented exactly	use `long` cents, or `BigDecimal`
`char c = 65;` then `c + 1`	gives `int` 66, not a `char`	cast: `(char) (c + 1)`
Assigning a `long` literal without `L`	`int` literal overflow error	write `9_000_000_000_000L`

- Print a value you do not trust: `System.out.println` on a small example beats guessing.
- Give every numeric field a unit in its name: `cents`, `millis`, `meters`.

2.5 Practice exercises

1. Predict and then check the output of `System.out.println(5 + " " + 5 / 2 + " " + 5.0 / 2 + " " + 'A' + 1);`
2. Why does `short s = 4; s = s + 1;` fail to compile while `s += 1;` works?
3. Write a text block that contains two lines of JSON without any `\n`.
4. How do you compare the content of two strings safely when one may be `null`?
5. Which type would you use for the number of milliseconds since 1970, and why?

Solutions

1. `5 2 2.5 66` - the division is integer division, `5.0 / 2` is `2.5`, and `'A' + 1` is `int` arithmetic giving 66 (no concatenation yet).
2. `s + 1` promotes to `int`, and the compiler refuses a silent narrowing cast; the compound assignment `s += 1` hides the cast, so it compiles.
3. `java String json = """ {"a": 1, "b": 2}""";` The common indentation is removed and no escape is needed.
4. `java.util.Objects.equals(a, b)` returns `true` when both are `null`, or `a.equals(b)`; writing `a.equals(b)` directly throws if `a` is `null`.
5. `long`, because milliseconds since the epoch already exceed the `int` range, and the suffix `L` marks the literal.

2.6 Mini test

1. Which is not a primitive type? A) `int` B) `double` C) `String` D) `boolean`
2. What is the type of `7 / 2`? A) `double` B) `float` C) `int` D) `long`
3. What does `var` change about a local variable? A) It makes it dynamically typed B) Nothing: the type is inferred and still fixed C) It allows reassignment to another type D) It makes the field visible everywhere
4. Which expression gives the character `'B'`? A) `'A' + 1` B) `(char) ('A' + 1)` C) `'A' + '1'` D) `65 + ""`
5. True or false: `"abc".equals(new String("abc"))` is `true`.
6. Which literal is a valid `long`? A) `3_000_000_000` B) `3000000000L` C) `3000000000.0` D) `3e9f`
7. What does the closing `"""` of a text block decide? A) The character set B) How much common indentation is removed C) The number of lines D) Nothing, it is decoration

Answer key and explanations

1. C - `String` is a class; it looks like a primitive because of the language's special support for the `+` operator.
2. C - both operands are `int`, so the result is `int` and the fraction is discarded.
3. B - `var` is compile time inference; the declared type is fixed.
4. B - `'A' + 1` is an `int` with value 66, and the cast reinterprets it as a character.

5. True - `equals` compares content, and `String` overrides it.
6. B - the `L` suffix makes the literal a `long`; without it the value exceeds the `int` range and compilation fails.
7. B - the closing delimiter's indentation is the minimum stripped from every line of the block.